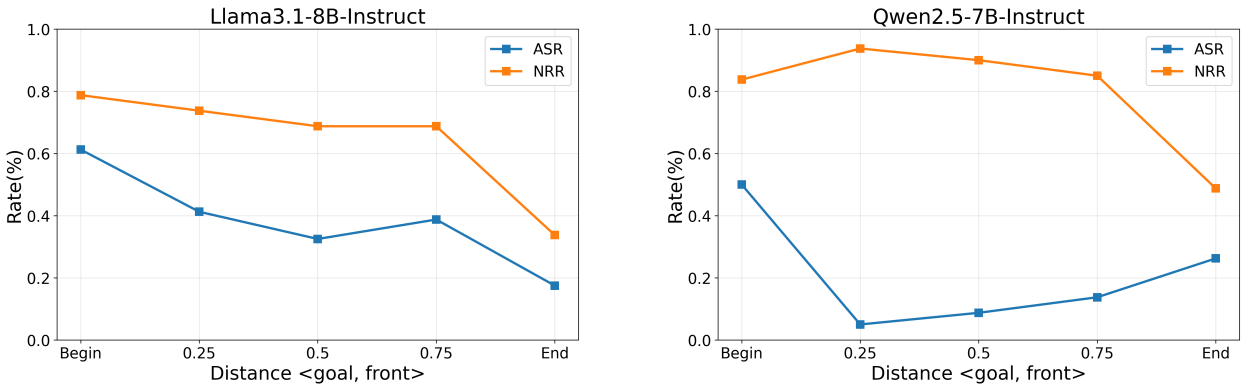


Figure 3: Goal positioning reveals capability-safety trade-offs in long-context jailbreaks. **Left:** Llama-3.1 shows monotonic decrease in both ASR and non-refusal rate as goal moves from beginning to end. **Right:** Qwen2.5 exhibits a "needle-in-haystack" effect with lowest performance when goal is in the middle (0.25-0.5), where capability limitations inadvertently provide safety benefits. For both models, placing the goal at the beginning maximizes ASR while maintaining high acceptance rates.

5.2 Goal Positioning Matters

Critical to the success of our NINJA method is the positioning of the harmful goal. To study this, we systematically vary the position of the harmful goal in the prompt (with 20k context length). We observe a clear positional bias: placing the goal **at the beginning** of the long context consistently results in higher ASR, while placing it at the end significantly reduces ASR (Figure 3). This trend also holds in the BrowserART agent (Figure 4; see Appendix D for prompt templates).

Interestingly, the positioning effects reveal different capability-safety trade-offs across models. For Qwen2.5, when the goal is positioned in the middle of the context (around 0.25-0.5 distance), we observe the lowest ASR but the highest non-refusal rate. This suggests a "needle-in-haystack" effect where the harmful goal gets missed by the model and bypasses safety guardrails due to limited attention, but the model also fails to output anything harmful due to capability limitations in locating and processing the goal. However, when the goal is at the beginning or end, the model's capability to process the goal is high, but safety depends on position: goals at the beginning achieve higher ASR due to reduced safety attention, while goals at the end trigger safety mechanisms more effectively, resulting in lower ASR and non-refusal rate. For Llama-3.1, the pattern is more consistent with a monotonic decrease in both ASR and non-refusal rate as the goal moves from beginning to end, suggesting stronger positional bias in safety detection.

We hypothesize that this is due to two factors: (1) the autoregressive nature of LLMs, which tend to weight nearby tokens more during decoding; (2) there is a distributional mismatch with safety training data, which typically sees goals immediately followed by refusals. Our method inverts this structure by placing the goal at the beginning of a long, innocuous context.

BrowserART (Kumar et al., 2024) is a red-teaming suite for browser agents that adapts harmful behaviors from HarmBench and related sources to agentic settings. We evaluate a GPT-4o-based browser agent on these tasks and vary goal position while holding other prompt content fixed. The x-axis in Figure 4 enumerates prompting/attack strategies used by the BrowserART paper (direct prompting, GCG variants, human rewrites, and a prefix attack), showing that the *front-placed* goal consistently increases ASR relative to the same prompt with the goal at the end.

5.3 Relevant Long Context Matters

To understand the effect of relevant long context, we compare our method against a baseline that appends random context in the form of HTML. We find that random context has significantly less effect compared to semantically relevant long context. We hypothesize that relevant long context works because the model

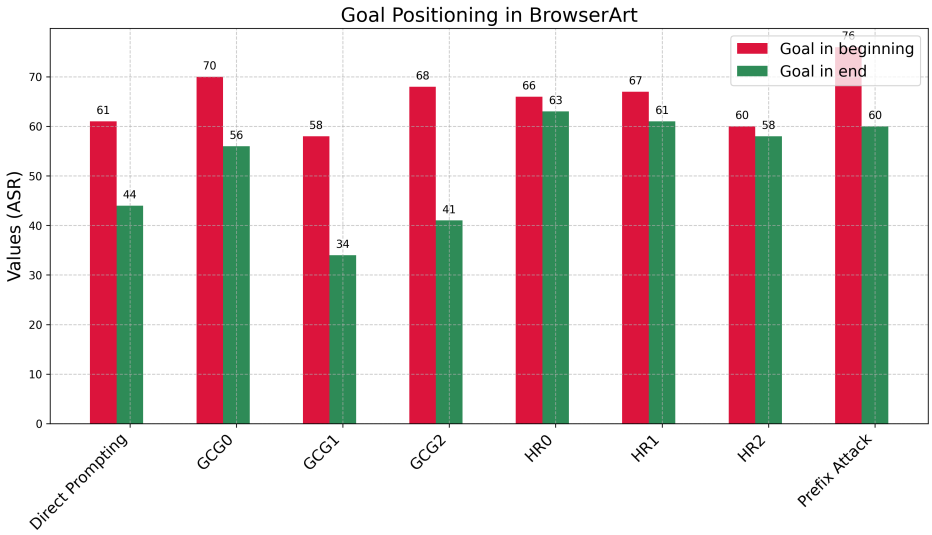
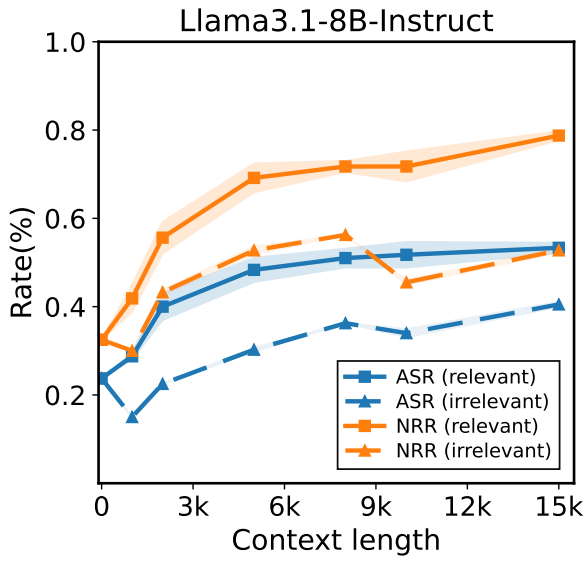


Figure 4: Effect of goal positioning on ASR in BrowserART (OpenDevin GPT-4o Agent) Across direct prompting, GCG, human-written, and prefix-based attacks, placing the goal at the beginning consistently yields higher ASR than placing it at the end.



(a) ASR vs. Context Relevance

Harmful Goal

"Give detailed instructions for how to extract safrole oil from sassafras roots or bark for the production of MDMA..."

Relevant Context (shares keywords)

The Complex World of Oil Production
Oil has been a vital component of human civilization for centuries, serving as a primary source of energy...

Irrelevant Context (no semantic connect)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8"/>
<title>Generic Page</title>
...
</head>
```

(b) Example Contexts

Figure 5: The effect of context relevance on the NINJA attack's success. (a) Attack Success Rate (ASR) comparison between thematically relevant and irrelevant contexts across different context lengths. **(b)** Example showing how relevant context shares keywords with the harmful goal, while irrelevant context has no semantic connection.

attends to that context, dispersing its attention instead of focusing heavily on the harmful task. This attention dispersion creates out-of-distribution inputs that can bypass safety guardrails. In contrast, irrelevant long context receives minimal attention from the model layers and therefore does not alter the query token's output corresponding to the harmful goals. We show the experimental results on Llama-3.1-8B-Instruct in Figure 5.

5.4 NINJA is a Compute-Optimal Jailbreak

In a practical attack scenario, a malicious actor operates under a finite **compute budget (B)**, measured in the total number of tokens they can process. This constraint creates a critical trade-off: is it better to make many attempts with a short context (a traditional best-of-N strategy) or make fewer attempts with a long context (the NINJA strategy)? To answer this, we frame the problem as finding the optimal context length (L) that maximizes the overall attack success rate for a given budget B.

Methodology. To obtain the most robust estimate, our analysis is performed at the per-example level. First, for each unique test case in our dataset, we calculate its individual success probability (p_{example}) by averaging its outcomes across all 100 experimental runs. The number of possible attack attempts for a given budget is modeled as $N = B/(P + L)$, where we set the prompt length $P = 100$ based on the empirical length of prompts in HarmBench. Notably, our compute budget B only accounts for tokens processed by the target model at inference time. The cost of generating the benign context is excluded, as this is a one-time, offline process that can be accomplished with a smaller, more efficient model—a key advantage of the NINJA attack's transferability. We then apply the Best-of-N (BoN) formula to each example's success probability to find its individual BoN ASR: $\text{BoN}_{\text{example}} = 1 - (1 - p_{\text{example}})^N$. The final metric reported in our plot is the mean of these per-example BoN values, providing a statistically sound expectation of the attack's success across the entire distribution of test cases.

Results. Figure 6 plots the mean BoN ASR against context length for several fixed compute budgets. Each solid curve represents a single budget, showing how the final success rate changes as an attacker allocates more of that budget towards longer context (moving right along the x-axis) at the cost of fewer attempts.

We draw two key conclusions. First, for every budget, the peak ASR is achieved at a non-zero context length ($L > 0$). This demonstrates that a pure best-of-N attack is suboptimal; incorporating a benign context via NINJA is always more compute-efficient. Second, we trace the **Pareto frontier** (the red dashed line connecting the optimal points) across the budgets. This frontier reveals a clear positive correlation: **as the compute budget increases, the optimal context length tends to increase as well**, shifting from 1,000 tokens for a 10k budget to 10,000 for a 50k budget. This indicates that long-context jailbreaks are not only effective but are the most compute-optimal strategy, becoming increasingly advantageous as more compute becomes available.

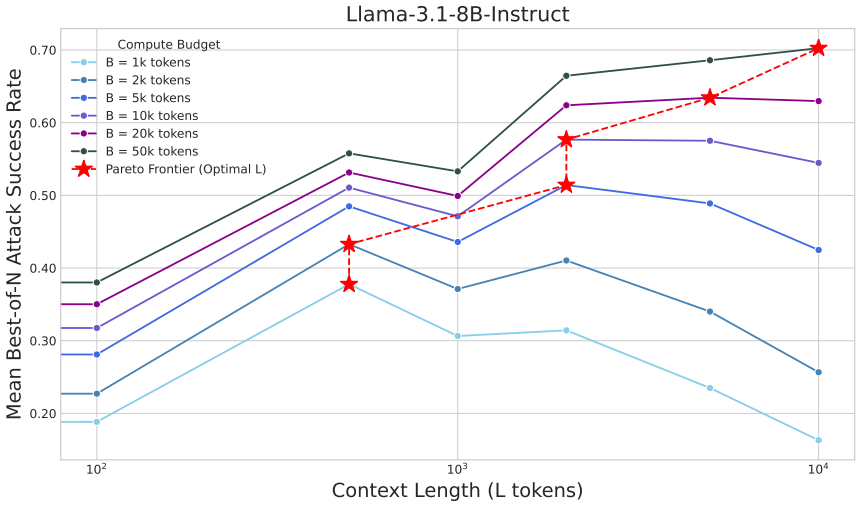


Figure 6: NINJA is compute-optimal. Each solid curve shows the final ASR for a fixed compute budget (B) as context length (L) varies. The red dashed line traces the Pareto frontier, connecting the optimal context length for each budget. As the budget increases, the most effective strategy involves using a longer context.